MRDVS® 迈尔微视

# LxCameraSDK (Python)

# Developer Guide

Version: V2.0

Release Date: 2026.06.04

GitHub: https://github.com/MRDVS/CameraSDK

Knowledge Base: https://hub.mrdvs.cn/

Official Website: https://www.mrdvs.cn/&https://mrdvs.com/

# Contents

# 1. Overview

This document is intended for Python developers who use LxCameraSDK to connect MRDVS/LxCamera series cameras. It describes how to install the Python package, load dynamic libraries, search for and connect devices, configure parameters, acquire streams synchronously, register status callbacks, read image and point cloud data, and develop with built-in application algorithms such as obstacle avoidance, pallet docking, and vision localization.

The Python interface loads the underlying LxCameraApi dynamic library through ctypes and converts part of the pointer data into Python objects or numpy.ndarray objects. During development, pay attention to both the Python package version and the version of the actually loaded LxCameraApi.dll or libLxCameraApi.so.

Table 1-1 SDK Capability Scope

| Capability | Description |
|---|---|
| Device access | Supports searching the device list and opening devices by index, IP, SN, or ID. After a device is opened successfully, a DcHandle is returned. All subsequent device operations depend on this handle. |
| Python wrapper | The LxCamera class wraps the underlying C interfaces. Enumerations, structures, and callback types are located in lx_camera_define.py and lx_camera_application.py. |
| Parameter configuration | LX_CAMERA_FEATURE uses the LX_INT, LX_FLOAT, LX_BOOL, LX_STRING, LX_CMD, and LX_PTR prefixes to distinguish read/write modes. The Python interface requires an LX_CAMERA_FEATURE enumeration object as input. |
| Image streaming | Supports 2D/RGB, 3D depth, and 3D amplitude streams. In synchronous mode, getFrame triggers LX_CMD_GET_NEW_FRAME and then reads different types of image data from FrameInfo. |
| Point cloud output | getPointCloud reads LX_PTR_XYZ_DATA and converts it to a numpy.ndarray with shape (height, width, 3). DcSaveXYZ supports saving in txt, ply, and pcd formats. |
| Status callback | Supports registering camera status callbacks. CameraStatus is returned when the status changes. The current Python package does not wrap frame callback or IMU callback functions. |
| Built-in algorithms | Supports algorithm modes such as MODE_AVOID_OBSTACLE, MODE_AVOID_OBSTACLE2, MODE_PALLET_LOCATE, and MODE_VISION_LOCATION. Algorithm results are read through getAlgorithmStatus. |

Table 1-2 Current Version Information

| Item | Content |
|---|---|
| Python package version | lx-camera-py 1.3.3. The whl file is located at Sample/python/lx_camera_py-1.3.3-py3-none-any.whl. |

| Item | Content |
|---|---|
| Python wrapper SDK_VERSION | 2.4.30. It is defined in LxCameraSDK/lx_camera_api.py and is used for runtime version checking. |
| Underlying API version macro | LX_CAMERA_API_VERSION V2.4.60,20260126. It is defined in SDK/include/lx_camera_api_version.h. |
| Dependencies | numpy and opencv-python. The examples additionally use open3d for point cloud visualization, but open3d is not a required dependency of the Python package. |
| Main sample directory | Sample/python/lx_camera_py/LxCameraSDK/Sample. |

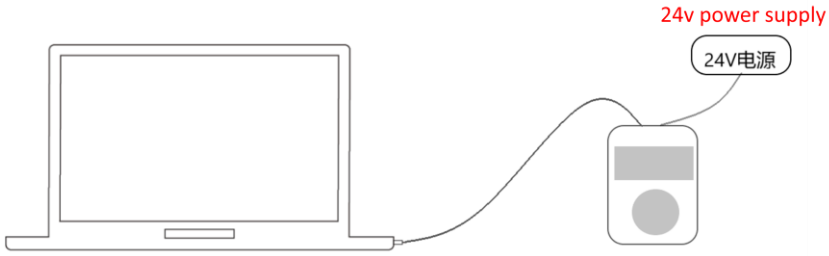
# 2. Environment Setup

## 2.1 Recommended Host System Configuration

| Configuration Item | Recommended Configuration | Description |
|---|---|---|
| Operating system | Windows7/10/11; Ubuntu16.04 or later | The Python package can be used on Windows and Linux. The dynamic library suffixes are .dll and .so respectively. In Linux Arm environments, confirm that the installation package and runtime library match the architecture. |
| Python version | Python3.x | 64-bit Python is recommended to match the win_x64 dynamic library. If 32-bit Python is used, load the win_x86 dynamic library. |
| CPU | 4 cores or more | For multiple cameras, point cloud conversion, OpenCV display, and application algorithm output, a higher frequency CPU or more cores are recommended. |
| Memory | 4GB or more | RGBD, point cloud saving, and multi-camera caching consume considerable memory. |
| Network adapter | Gigabit or higher | When depth, amplitude, and RGB streams are enabled at the same time, an independent Gigabit Ethernet port is recommended to avoid sharing bandwidth with other high-bandwidth services. |
| Disk | Reserve space for logs and point cloud saving | Saving pcd/ply/txt point clouds may quickly consume disk space. Pay attention to cleanup during debugging. |

## 2.2 Hardware Environment Setup

Connect the camera to an external 24V power supply and connect it to the host computer through a network cable, as shown in the figure. RGBD, point cloud saving, and multi-camera caching consume considerable memory.




| Check Item | Requirement |
|---|---|
| Power supply | Confirm that the voltage, current, and wiring of the 24V power supply meet the camera requirements. Avoid device reboot or stream interruption caused by unstable power supply. |
| Network cable | Use a reliable network cable and confirm that the host network port negotiates to gigabit speed. The negotiated network speed can be obtained through LX_INT_LINK_SPEED. |
| IP planning | The default camera subnet is usually 192.168.100.*. The host network port must be configured to the same subnet, and the address must not conflict. |
| Multi-camera connection | Avoid duplicate IP addresses among multiple cameras. When TOF cameras interfere with each other, handle it together with LX_BOOL_ENABLE_MULTI_MACHINE and the on-site installation angle. |

## 2.3 Network Environment Setup

The default camera IP subnet is usually 192.168.100.*. When the host computer is directly connected to the camera, set the IPv4 address of the corresponding network port to the same subnet, for example, 192.168.100.99, with subnet mask 255.255.255.0. If the camera IP has been changed, use LxDeviceInfo.ip returned by DcGetDeviceList as the reference.


When connecting a camera, it is recommended to disable the firewall to prevent camera packets from being intercepted by the host running the SDK, as shown in the figure.


| Platform | Recommended Operation |
|---|---|
| Windows | Configure the IPv4 address of the network port to the same subnet as the camera. During debugging, temporarily disable the firewall or allow network access for the Python interpreter and SDK sample programs. |

| Platform | Recommended Operation |
| --- | --- |
| Ubuntu | Configure the network port in the same way as Windows. For the firewall, use sudo systemctl stop ufw.service, sudo systemctl disable ufw.service, and sudo ufw status to check it. |
| Cross-subnet | SDK discovery depends on network reachability. In cross-subnet or multi-NIC environments, first confirm the route, gateway, subnet mask, and actual camera IP. |
| Change the camera IP | After DcSetCameraIp succeeds, the camera restarts automatically and the device list changes. DcGetDeviceList must be called again before opening the device. |

## 2.4 Software Environment Setup

| Item | Description |
| --- | --- |
| Installation package | Sample/python/lx_camera_py-1.3.3-py3-none-any.whl. |
| Source package | Sample/python/lx_camera_py. It contains the LxCameraSDK package, samples, and dist-info metadata. |
| Required dependencies | numpy and opencv-python. opencv-python is mainly used for sample image display and normalized pseudo-color display. |
| Optional dependency | open3d is used only for the point cloud visualization example in single_camera.py and does not affect basic SDK calls. |
| Dynamic library | Windows loads LxCameraApi.dll. Linux loads libLxCameraApi.so. The Python package does not automatically search the SDK directory. The dynamic library path must be passed in when LxCamera is instantiated. |

```
# Windows example
cd /d D:\Program Files\MRDVS\Sample\python
python -m pip install lx_camera_py-1.3.3-py3-none-any.whl

# Linux example
cd /opt/MRDVS/Sample/python
python3 -m pip install lx_camera_py-1.3.3-py3-none-any.whl
```

If the runtime environment cannot access the Internet, prepare offline installation packages for dependencies such as numpy, opencv-python, and open3d in advance. If only SDK interfaces are called and images do not need to be displayed, install the Python package and numpy first, and then install OpenCV-related dependencies as needed.

# 3. SDK Directory and Project Configuration

## 3.1 LxCameraSDK Directory Structure

| Directory | Content |
|---|---|
| Document | SDK documentation, developer guides, sample manuals, and FAQs. |
| FirmWare | Camera firmware upgrade packages. |
| Sample | Sample code for different languages and functional scenarios. Python samples are located in Sample/python. |
| Sample/python | Python whl installation package, source package, and packaging scripts. |
| SDK/include | C/C++ header files. Enumerations and structures in the Python wrapper correspond to these header files. |
| SDK/lib | Windows dynamic libraries and link libraries. The Python runtime needs to load LxCameraApi.dll for the corresponding architecture. |

## 3.2 Python Package Directory Structure

| File or Directory | Description |
|---|---|
| LxCameraSDK/__init__.py | Exports the main objects in lx_camera_define, lx_camera_api, and lx_camera_application. These definitions are loaded when from LxCameraSDK import * is used. |
| LxCameraSDK/lx_camera_api.py | Defines the LxCamera class, loads the dynamic library, initializes underlying C function signatures, and provides Python calling methods. |
| LxCameraSDK/lx_camera_define.py | Defines common enumerations, device information structures, frame structures, callback types, and the LX_CAMERA_FEATURE parameter enumeration. |
| LxCameraSDK/lx_camera_application.py | Defines structures related to application algorithms such as obstacle avoidance, pallet docking, and localization. |
| LxCameraSDK/Sample/single_camera.py | Sample for single-camera synchronous streaming, status callback, image display, point cloud reading, and frame rate reading. |
| LxCameraSDK/Sample/application_obstacle.py | Sample for obstacle avoidance algorithm mode setting, algorithm parameter reading, and algorithm output reading. |
| LxCameraSDK/Sample/application_pallet.py | Sample for pallet docking algorithm mode setting and LxPalletPose result reading. |
| LxCameraSDK/Sample/application_location.py | Reference for vision localization-related structures. The current source code is marked TODO NOT FINISHED, DO NOT USE IT. It must be completed and verified before use in formal projects. |

| File or Directory | Description |
| --- | --- |
| LxCameraSDK/Sample/utils.py | Helper functions for image visualization and LX_STATE checking. |

## 3.3 Installation and Runtime Library Configuration

After the Python package is installed, the underlying dynamic library is loaded through LxCamera(dll_path) in code. In Windows environments, make sure the Python architecture matches the dynamic library architecture. In Linux environments, make sure libLxCameraApi.so and its dependent libraries can be loaded by the system.

```
from LxCameraSDK import *

# Windows
camera = LxCamera(r"D:\Program Files\MRDVS\SDK\lib\win_x64\LxCameraApi.dll")

# Linux
# camera = LxCamera("/opt/MRDVS/lib/libLxCameraApi.so")
```

| Platform | Dynamic Library Path Example | Notes |
| --- | --- | --- |
| Windows x64 | D:\Program Files\MRDVS\SDK\lib\win_x64\LxCameraApi.dll | When using 64-bit Python, load the DLL in the win_x64 directory. If necessary, add SDK/lib/win_x64 to PATH, or place related DLLs in a directory accessible to the program. |
| Windows x86 | D:\Program Files\MRDVS\SDK\lib\win_x86\LxCameraApi.dll | When using 32-bit Python, load the DLL in the win_x86 directory. |
| Linux | /opt/MRDVS/lib/libLxCameraApi.so | After install.sh is executed, LD_LIBRARY_PATH is usually set. In a new terminal, source ~/.bashrc. For non-root users, confirm the environment variables of the current user. |

## 3.4 Minimal Project Configuration

A minimal Python project does not require CMake or a Makefile. It only needs to ensure that the Python environment, LxCameraSDK package, and underlying dynamic library path are correct. It is recommended to place the camera IP, dynamic library path, log path, enabled streams, and other configuration items in a configuration file or at the beginning of the script for easier on-site debugging.

| File | Purpose |
|---|---|
| main.py | Main process for device search, opening, stream starting, frame acquisition, and closing. |
| config.py or config.json | Stores the camera IP, dynamic library path, log directory, enabled data streams, and other configuration. |
| utils.py | Wraps check_state, image display, log output, and resource release logic. |
| requirements.txt | Records dependencies such as numpy, opencv-python, and open3d. |

# 4. Development Workflow

## 4.1 General Interface Calling Principles

The Python wrapper performs type checking on the cmd parameter. Pass an LX_CAMERA_FEATURE enumeration object. Passing an integer value directly is not recommended. DcSetIntValue supports both a normal int and an Enum object as value. If value is an Enum, its .value is taken automatically.

| Feature Prefix | Interface | Purpose |
|---|---|---|
| LX_INT | DcSetIntValue / DcGetIntValue | Read and write integer parameters, such as exposure, image size, trigger mode, algorithm mode, and filter level. |
| LX_FLOAT | DcSetFloatValue / DcGetFloatValue | Read and write floating-point parameters, such as filter strength, light intensity, frame rate, and temperature. |
| LX_BOOL | DcSetBoolValue / DcGetBoolValue | Reads and writes Boolean switches, such as stream enable switches, auto exposure, undistortion, and synchronized frames. |
| LX_STRING | DcSetStringValue / DcGetStringValue | Reads and writes string parameters, such as firmware name, algorithm parameters, algorithm version, and parameter import/export paths. |
| LX_CMD | DcSetCmd/getFrame | Execute commands, such as refreshing frames, software trigger, device restart, and restoring default parameters. |
| LX_PTR | DcGetPtrValue | Obtains internal data pointers. Common use cases have been wrapped as getFrame, getPointCloud, get2DIntricParam, get3DIntricParam, getAlgorithmStatus, and other methods. |

## 4.2 Synchronous Interface Call Workflow

| Step | Interface/Operation | Description |
|---|---|---|
| 1 | LxCamera(dll_path) | Loads the underlying LxCameraApi dynamic library, initializes C function signatures, and checks the runtime version. |
| 2 | DcSetInfoOutput | Sets the log level, whether to output to the screen, log directory, and language as needed. |
| 3 | DcGetDeviceList | Searches devices and returns state, dev_list, and dev_num. |
| 4 | DcOpenDevice | Opens a camera by IP, SN, ID, or index, and returns handle and LxDeviceInfo. |
| 5 | DcSetBoolValue | Enables 2D, 3D depth, and 3D amplitude streams as needed. |
| 6 | DcGetIntValue | Reads image width, height, channel count, and data type. After ROI, Binning, or RGBD alignment, read them again. |
| 7 | DcStartStream | Starts streaming. |
| 8 | getFrame | Internally calls LX_CMD_GET_NEW_FRAME and then reads LX_PTR_FRAME_DATA. |
| 9 | getDepthImage/getAmpImage/getRGBImage | Converts corresponding image data from FrameInfo to numpy.ndarray. |
| 10 | getPointCloud/DcSaveXYZ | Reads XYZ point cloud or saves a point cloud file. |
| 11 | DcStopStream/DcCloseDevice | Stops streaming and closes the device before exit. |

```
from LxCameraSDK import *

camera = LxCamera(r"D:\Program Files\MRDVS\SDK\lib\win_x64\LxCameraApi.dll")
state, dev_list, dev_num = camera.DcGetDeviceList()
if state != LX_STATE.LX_SUCCESS or dev_num <= 0:
    raise RuntimeError("No camera found")

state, handle, device_info = camera.DcOpenDevice(LX_OPEN_MODE.OPEN_BY_IP,
"192.168.100.82")
if state != LX_STATE.LX_SUCCESS:
    raise RuntimeError(camera.DcGetErrorString(state))

camera.DcSetBoolValue(handle, LX_CAMERA_FEATURE.LX_BOOL_ENABLE_3D_DEPTH_STREAM, True)
camera.DcSetBoolValue(handle, LX_CAMERA_FEATURE.LX_BOOL_ENABLE_3D_AMP_STREAM, True)
camera.DcSetBoolValue(handle, LX_CAMERA_FEATURE.LX_BOOL_ENABLE_2D_STREAM, True)
```

```
camera.DcStartStream(handle)


state, frame = camera.getFrame(handle)
if state == LX_STATE.LX_SUCCESS:
    state, depth = camera.getDepthImage(frame)
    state, amp = camera.getAmpImage(frame)
    state, rgb = camera.getRGBImage(frame)
camera.DcStopStream(handle)

camera.DcCloseDevice(handle)
```

## 4.3 Status Callback Development Workflow

The current Python package wraps the camera status callback, but does not wrap the frame callback and IMU callback available in the C/C++ version. The status callback is suitable for monitoring state changes such as camera opened, stream started, disconnection, and reconnection. Image acquisition still uses getFrame in synchronous mode. Note: the current Python wrapper creates a ctypes callback object inside DcRegisterCameraStatusCallback, but does not explicitly save the object reference. If status callbacks need to be used for a long time in a formal project, it is recommended to save the callback object in the wrapper layer to prevent it from being garbage-collected by Python.

| Step | Interface/Operation | Description |
|---|---|---|
| 1 | Define callback(state_ptr) | The callback parameter is a CameraStatus pointer. camera_state and status_time can be read through state_ptr.contents. |
| 2 | DcRegisterCameraStatusCallback | Registers the status callback after the device is opened. |
| 3 | Acquire data or wait for status changes | Status changes such as disconnection, reconnection, and stream start are triggered by the SDK. |
| 4 | DcUnregisterCameraStatusCallback | Unregisters the callback before closing the device to avoid invalid callback function references. |

```
def camera_status_callback(status_ptr):

    status = status_ptr.contents

    print(status.camera_state, status.status_time)


state = camera.DcRegisterCameraStatusCallback(handle, camera_status_callback)

# Before exit

state = camera.DcUnregisterCameraStatusCallback(handle)
```

### 4.4 Built-in Application Algorithm Workflow

| Step | Interface/Operation | Description |
|---|---|---|
| 1 | DcSetIntValue(LX_INT_ALGORITHM_MODE, mode) | Selects MODE_AVOID_OBSTACLE, MODE_AVOID_OBSTACLE2, MODE_PALLET_LOCATE, or MODE_VISION_LOCATION. |
| 2 | DcGetStringValue(LX_STRING_ALGORITHM_VERSION) | Reads the version corresponding to the current algorithm mode. The algorithm mode must be set first. |
| 3 | DcGetStringValue / DcSetStringValue(LX_STRING_ALGORITHM_PARAMS) | Reads or writes algorithm parameter JSON. Before writing, it is recommended to use json.loads to check the format, and then use json.dumps to write it back. |
| 4 | DcSetIntValue(LX_INT_WORK_MODE, WORK_FOREVER) | Some application algorithm samples set always-on mode. In actual projects, confirm this according to the camera model and on-site logic. |
| 5 | DcStartStream/getFrame | After streaming starts, acquire frames in a loop to make sure algorithm output data is refreshed. |
| 6 | getAlgorithmStatus | Automatically converts the output to LxAvoidanceOutput, LxAvoidanceOutputN, LxPalletPose, or LxLocation according to the current algorithm mode. |
| 7 | DcSpecialControl | Special control commands are used for extended operations such as GetObstacleIO, SetLaserData, and ImportLocationMapFile. Commands and parameters must be confirmed according to the specific algorithm. |

# 5. Notes on Interface Calls

## 5.1 Call Sequence and Resource Management

| Rule | Description |
|---|---|
| Open and close in pairs | After DcOpenDevice succeeds, DcCloseDevice must be called to release resources and exclusive control permission. If the program exits abnormally without closing the device, wait until the heartbeat times out before the permission is released. |
| Start and stop streaming in pairs | After DcStartStream succeeds, call DcStopStream before exit. Close the device after streaming stops. |
| Unregister callbacks first | When a status callback is used, call DcUnregisterCameraStatusCallback before closing the device. |
| Synchronous refresh | getFrame internally executes LX_CMD_GET_NEW_FRAME. If only point cloud data is read, call DcSetCmd or getFrame first to refresh data. |

| Rule | Description |
| --- | --- |
| Do not release internal memory | Internal memory returned by DcGetDeviceList, DcGetStringValue, and DcGetPtrValue is usually managed by the SDK. When data needs to be stored across threads or for a long time, copy it to Python objects or numpy arrays. |
| Fixed dynamic library path | LxCamera initialization does not automatically search the SDK installation directory. dll_path must point to a dynamic library that can be loaded on the current platform. |

## 5.2 Preconditions for Some Interface Calls

| Category | Restriction or Precondition |
| --- | --- |
| Streaming state restrictions | When streaming is active or LX_INT_WORK_MODE is WORK_FOREVER, do not set parameters that must be determined before streaming starts, such as ROI, Binning, trigger mode, calculation offload, and multi-camera mode. |
| Auto exposure | When 3D auto exposure is enabled, manual multi-exposure HDR, exposure value, and gain cannot be set. When 2D auto exposure is enabled, 2D manual exposure and manual gain should not be set. |
| Image size | After ROI, Binning, or RGBD alignment is set, the 3D/2D image width, height, channel count, and data type may change. Call DcGetIntValue again to obtain them. |
| Point cloud reading | Before reading point clouds, LX_BOOL_ENABLE_3D_DEPTH_STREAM must be enabled, and a valid frame must have been refreshed through DcSetCmd or getFrame. |
| Algorithm parameters | Before reading or writing LX_STRING_ALGORITHM_PARAMS or LX_STRING_ALGORITHM_VERSION, LX_INT_ALGORITHM_MODE must be set first. |
| Status callback | The callback function object must remain valid during registration. Do not allow it to be released early in a local scope. |
| ROI interface | The underlying C interface name is DcSetROI. In the current Python source code, _init_c_func configures self._dll.DcSetROI, but the wrapper method DcSetRoI actually calls self._dll.DcSetRoI. The uppercase/lowercase spelling is inconsistent. Before formally using ROI, fix the Python wrapper so that the method calls self._dll.DcSetROI, or extend the corresponding ctypes call by yourself. |

## 5.3 Return Value Handling

For most LxCamera methods, the first element of the return value is LX_STATE. LX_STATE.LX_SUCCESS indicates success. Other states can be queried through DcGetErrorString for underlying error descriptions. In Python projects, it is recommended to wrap a unified check_state function to avoid repeated error checks at every call site.

```
def check_state(camera, state, handle=None):
```

```
    if state != LX_STATE.LX_SUCCESS:
        if state == LX_STATE.LX_E_RECONNECTING:
            print("device reconnecting")
            return
        print(camera.DcGetErrorString(state))
        if handle is not None:
            camera.DcCloseDevice(handle)
        raise RuntimeError(state)
```

# 6. Data Types and Enumerations

The data types, enumerations, and feature parameters in this chapter are mainly consistent with the underlying C/C++ interfaces of LxCameraSDK. The Python package wraps the underlying dynamic library through ctypes. The current lx_camera_define.py explicitly defines only common enumerations, structures, and parameter items. Therefore, some enumerations or parameters supported by the underlying SDK may not be declared in the current Python package. If a parameter is not declared in the current Python wrapper, do not directly pass a raw integer to interfaces such as DcSetIntValue or DcGetIntValue, because the wrapper code checks whether cmd is an LX_CAMERA_FEATURE enumeration object. Instead, add the corresponding enumeration to the Python wrapper first, or extend the ctypes call according to the underlying C API.

## 6.1 LX_STATE

Status codes used by SDK interface functions and frame status.

| Keyword | Value | Description |
|---|---|---|
| LX_SUCCESS | 0 | Success. |
| LX_ERROR | -1 | Unknown error. |
| LX_E_NOT_SUPPORT | -2 | Function not supported. |
| LX_E_NETWORK_ERROR | -3 | Network communication error. |
| LX_E_INPUT_ILLEGAL | -4 | Invalid input parameter. |
| LX_E_RECONNECTING | -5 | Device reconnecting. |
| LX_E_DEVICE_ERROR | -6 | Device fault or device response failure. |
| LX_E_DEVICE_NEED_UPDATE | -7 | Device version is too low and must be upgraded. |
| LX_E_API_NEED_UPDATE | -8 | API version is too low. |

| Keyword | Value | Description |
|---|---|---|
| LX_E_CTRL_PERMISS_ERROR | -9 | Exclusive control permission failed. |
| LX_E_GET_DEVICEINFO_ERROR | -10 | Failed to obtain device information. |
| LX_E_IMAGE_SIZE_ERROR | -11 | Image size mismatch. Reopen the device. |
| LX_E_IMAGE_PARTITION_ERROR | -12 | Image parsing failed because pixformat is incorrect. Try reopening the device. |
| LX_E_DEVICE_NOT_CONNECTED | -13 | Camera is not connected. |
| LX_E_DEVICE_INIT_FAILED | -14 | Camera initialization failed. |
| LX_E_DEVICE_NOT_FOUND | -15 | Camera not found, or matching camera not found. |
| LX_E_FILE_INVALID | -16 | File error. The file name, type, or format is incorrect, or the file failed to open. |
| LX_E_CRC_CHECK_FAILED | -17 | File CRC or MD5 check failed. |
| LX_E_TIME_OUT | -18 | Timeout. |
| LX_E_FRAME_LOSS | -19 | Frame loss. |
| LX_E_ENABLE_ANYSTREAM_FAILED | -20 | Failed to enable any stream. |
| LX_E_NOT_RECEIVE_STREAM | -21 | No stream data received. |
| LX_E_PARSE_STREAM_FAILED | -22 | Streaming started successfully but stream data parsing failed. |
| LX_E_PROCESS_IMAGE_FAILED | -23 | Image computation or processing failed. |
| LX_E_SETTING_NOT_ALLOWED | -24 | Setting is not allowed in always-on mode. |
| LX_E_LOAD_DATAPROCESSLIB_ERROR | -25 | Error loading image processing algorithm library. |
| LX_E_FUNCTION_CALL_LOGIC_ERROR | -26 | Function call logic error. |
| LX_E_IPAPPDR_UNREACHABLE_ERROR | -27 | IP unreachable or network configuration error. |
| LX_E_FRAME_ID_NOT_MATCH | -28 | Frame synchronization error within the timeout range. |
| LX_E_FRAME_MULTI_MACHINE | -29 | Multi-camera interference signal detected in the frame. |
| LX_E_FRAME_IMAGE_ERROR | -30 | Camera-device image-related exception detected in frame data. |
| LX_W_NOT_SUPPORT | 2 | Function not supported. |

| Keyword | Value | Description |
|---|---|---|
| LX_W_LOAD_DATAPROCESSLIB_ERROR | 25 | Failed to load image processing algorithm library. Other functions are not affected. |

## 6.2 LX_DATA_TYPE

Data formats for images, point clouds, and algorithm outputs.

| Keyword | Value | Description |
|---|---|---|
| LX_DATA_UNSIGNED_CHAR | 0 | Unsigned char data |
| LX_DATA_CHAR | 1 | Signed char data |
| LX_DATA_UNSIGNED_SHORT | 2 | Unsigned short data |
| LX_DATA_SIGNED_SHORT | 3 | Signed short data |
| LX_DATA_INT | 4 | Integer data |
| LX_DATA_FLOAT | 5 | Floating-point data |
| LX_DATA_DOUBLE | 6 | Double-precision floating-point data |
| LX_DATA_OBSTACLE | 16 | Obstacle avoidance algorithm structure type |
| LX_DATA_PALLET | 17 | Pallet docking algorithm structure type |
| LX_DATA_LOCATION | 18 | Visual localization algorithm structure type |
| LX_DATA_OBSTACLE2 | 19 | Obstacle avoidance algorithm V2 structure type |
| LX_DATA_CUSTOM | 100 | Camera application custom data type |

## 6.3 LX_DEVICE_TYPE

Device model enumeration. Actual supported capabilities depend on the specific device and firmware.

| Keyword | Value | Description |
|---|---|---|
| LX_DEVICE_M2 | 1001 | M2 series camera |
| LX_DEVICE_M3 | 1002 | M3 series camera |
| LX_DEVICE_M4Pro | 1003 | M4 Pro series camera |
| LX_DEVICE_M4_MEGA | 1004 | M4 MEGA series camera |
| LX_DEVICE_M4 | 1005 | M4 series camera |
| LX_DEVICE_M4V1_1 | 1006 | M4 V1.1 series camera |
| LX_DEVICE_M4ProV1_1 | 1007 | M4 Pro V1.1 series camera |
| LX_DEVICE_S1 | 2001 | S1 series camera |

| Keyword | Value | Description |
|---|---|---|
| LX_DEVICE_S2 | 2002 | S2 series camera |
| LX_DEVICE_S2MaxV1 | 2003 | S2 Max V1 series camera |
| LX_DEVICE_S2MaxV2 | 2004 | S2 Max V2 series camera |
| LX_DEVICE_S2MaxV1_1 | 2005 | S2 Max V1.1 series camera |
| LX_DEVICE_S3 | 2101 | S3 series camera |
| LX_DEVICE_S10 | 2108 | S10 series camera |
| LX_DEVICE_S10PRO | 2109 | S10 PRO series camera |
| LX_DEVICE_S10ULTRA | 2110 | S10 ULTRA series camera |
| LX_DEVICE_S11 | 2201 | S11 series camera |
| LX_DEVICE_I1 | 3001 | I1 series camera |
| LX_DEVICE_I2 | 3002 | I2 series camera |
| LX_DEVICE_T1 | 4001 | T1 series camera |
| LX_DEVICE_T2 | 4002 | T2 series camera |
| LX_DEVICE_H3 | 5001 | H3 series camera |
| LX_DEVICE_H4 | 5002 | H4 series camera |
| LX_DEVICE_V1Pro | 6001 | V1 Pro series camera |
| LX_DEVICE_V2Pro | 6002 | V2 Pro series camera |
| LX_DEVICE_NULL | 0 | Reserved or invalid device type |

## 6.4 LX_DEVICE_SERIALS

Device category search filter enumeration.

| Keyword | Value | Description |
|---|---|---|
| LX_SERIAL_GIGE | 1 | GigE network camera device category |
| LX_SERIAL_WK | 2 | WK device category |
| LX_SERIAL_OTHER | 3 | Other device category |
| LX_SERIAL_ALL | 4 | Search all device categories |
| LX_SERIAL_LOCAL_LOOP | 5 | Local loopback, used to distinguish the case where the SDK and the camera-side program run on the same host |

## 6.5 LX_OPEN_MODE

The meaning of the param parameter when opening a device depends on the open mode.

| Keyword | Value | Description |
|---|---|---|
| OPEN_BY_INDEX | 0 | Open by index in the search list. The corresponding parameter is the index number. When the discovered device list changes, the selected device may also change. |
| OPEN_BY_IP | 1 | Open by the corresponding IP in the search list. The corresponding parameter is device IP or ip:port. |
| OPEN_BY_SN | 2 | Open by the corresponding SN in the search list. The corresponding parameter is device SN. |
| OPEN_BY_ID | 3 | Open by the corresponding ID in the search list. The corresponding parameter is device ID. |

## 6.6 LX_2D_OUTPUT_FORMAT

2D image output format.

| Keyword | Value | Description |
|---|---|---|
| LX_2D_FORMAT_RAW | 0 | RAW original 2D image format |
| LX_2D_FORMAT_JPEG | 4 | In this mode, subsequent algorithm processing is not performed, including undistortion and alignment. |
| LX_2D_FORMAT_YUV_NV12 | 5 | YUV420 |
| LX_2D_FORMAT_YUV_UYVY | 6 | YUV422 |
| LX_2D_FORMAT_RGB | 8 | RGB888 |

## 6.7 JPEG_DECODE_METHOD

2D JPEG image decoding method.

| Keyword | Value | Description |
|---|---|---|
| JPEG_DECODE_TURBOJPEG | 0 | Turbo jpeg |
| JPEG_DECODE_DP | 1 | Use LxDataProcess |
| JPEG_DECODE_RKMPP | 2 | RK hardware decoding, dynamically loading librockchip_mpp.so |
| JPEG_DECODE_GPU | 3 | Not implemented yet |

## 6.8 LX_BINNING_MODE

Image pixel binning mode.

| Keyword | Value | Description |
|---|---|---|
| LX_BINNING_1X1 | 0 | 1x1 Binning, no pixel binning |
| LX_BINNING_2X2 | 1 | 2x2 pixel binning |

| Keyword | Value | Description |
|---|---|---|
| LX_BINNING_4X4 | 2 | 4x4 pixel binning |

## 6.9 LX_STRUCT_LIGHT_CODE_MODE

Structured-light camera coding mode.

| Keyword | Value | Description |
|---|---|---|
| LX_CODE_NORMAL | 1 | Normal |
| LX_CODE_STATBLE | 2 | Stable |
| LX_CODE_ENHANCE | 3 | Enhanced high precision |

## 6.10 LX_ALGORITHM_MODE

Camera built-in application algorithm mode.

| Keyword | Value | Description |
|---|---|---|
| MODE_ALL_OFF | 0 | Disable built-in obstacle avoidance algorithm |
| MODE_AVOID_OBSTACLE | 1 | Built-in obstacle avoidance algorithm |
| MODE_PALLET_LOCATE | 2 | Built-in pallet docking algorithm |
| MODE_VISION_LOCATION | 3 | Built-in visual localization algorithm |
| MODE_AVOID_OBSTACLE2 | 4 | Built-in obstacle avoidance algorithm V2 |
| MODE_GENERIC_DATA | 5 | Custom data type, application algorithm platform |

## 6.11 LX_CAMERA_WORK_MODE

Camera work mode.

| Keyword | Value | Description |
|---|---|---|
| KEEP_HEARTBEAT | 0 | Keep heartbeat. The camera automatically enters standby after the SDK heartbeat is interrupted. |
| WORK_FOREVER | 1 | Always keep working. Some parameters cannot be set in this mode. |

## 6.12 LX_TRIGGER_MODE

Trigger mode.

| Keyword | Value | Description |
|---|---|---|
| LX_TRIGGER_MODE_OFF | 0 | Trigger mode off. Stream mode. Default. |
| LX_TRIGGER_SOFTWARE | 1 | Software trigger mode |

| Keyword | Value | Description |
|---|---|---|
| LX_TRIGGER_HARDWARE | 2 | Hardware trigger mode |

## 6.13 LX_IO_WORK_MODE

IO work mode.

| Keyword | Value | Description |
|---|---|---|
| ALGORITHM_IO_MODE | 0 | IO is used as input/output for the built-in application algorithm. |
| USER_IO_MODE | 1 | The user can control the IO status. Refer to LX_IO_OUT_USERCTRL_MODE. |
| TRIGGER_IO_MODE | 2 | IO is used as trigger signal output. Valid only in trigger mode. |

## 6.14 LX_IO_OUTPUT_STATE

User-controlled IO output combination.

| Keyword | Value | Description |
|---|---|---|
| OUT1_0_OUT2_0 | 0 | OUT1 outputs 0, OUT2 outputs 0 |
| OUT1_0_OUT2_1 | 1 | OUT1 outputs 0, OUT2 outputs 1 |
| OUT1_1_OUT2_0 | 2 | OUT1 outputs 1, OUT2 outputs 0 |
| OUT1_1_OUT2_1 | 3 | OUT1 outputs 1, OUT2 outputs 1 |

## 6.15 LX_IO_HARDWARE_MODE

IO output hardware work mode.

| Keyword | Value | Description |
|---|---|---|
| OPEN_DRAIN_MODE | 0 | Open-drain normally-open mode |
| PUSH_PULL_MODE | 1 | Push-pull high-level mode |

## 6.16 LX_FILTER_MODE

Filter parameter mode.

| Keyword | Value | Description |
|---|---|---|
| FILTER_SIMPLE | 1 | Only one parameter: LX_FLOAT_FILTER_LEVEL |
| FILTER_NORMAL | 2 | Divided into multiple modules: smoothing, denoising, filling, and temporal filtering |
| FILTER_EXPERT | 3 | Configure detailed algorithms through strings |

## 6.17 LX_3D_FREQ_MODE

3D frequency mode.

| Keyword | Value | Description |
|---|---|---|
| FREQ_SINGLE_3D | 0 | Single frequency |
| FREQ_MULTI_3D | 1 | Multi-frequency |
| FREQ_TRIPLE_3D | 2 | Triple frequency |

## 6.18 LX_RGBD_ALIGN_MODE

RGBD alignment mode.

| Keyword | Value | Description |
|---|---|---|
| RGBD_ALIGN_OFF | 0 | Disable RGBD alignment. RGB and depth use their own resolutions and coordinate systems. |
| DEPTH_TO_RGB | 1 | Align depth to RGB. Resolution and coordinates are based on RGB, and RGB is forced to be undistorted. |
| RGB_TO_DEPTH | 2 | Align RGB to depth. Resolution and coordinates are based on depth. |
| DEPTH_TO_RGB_SPARSE | 3 | Computing-optimized DEPTH_TO_RGB. Resolution and coordinates are based on RGB, but the valid point cloud data amount is not changed. |

## 6.19 LX_CAN_PROTOCOL_TYPE

CAN protocol type.

| Keyword | Value | Description |
|---|---|---|
| CAN_MRDVS | 0 | MRDVS CAN protocol |
| CAN_KECONG | 1 | Kecong CAN protocol |

## 6.20 LX_POWER_MODE_TYPE

Camera power mode.

| Keyword | Value | Description |
|---|---|---|
| POWER_MODE_LOW | 0 | Low-power mode, with some functions disabled |
| POWER_MODE_DEFAULT | 1 | Default power mode, balancing performance and power consumption |
| POWER_MODE_HIGH | 2 | High-performance mode, enabling all functions |

| Keyword | Value | Description |
|---|---|---|
| POWER_MODE_STANDBY | 3 | Standby mode, with the lowest power consumption and image acquisition suspended |

## 6.21 LX_CAMERA_STATUS

Status enumeration in the camera status callback.

| Keyword | Value | Description |
|---|---|---|
| STATUS_CLOSED | 0 | Camera is not opened or is closed |
| STATUS_OPENED_UNSTARTED | 1 | Camera is opened but streaming has not started |
| STATUS_STARTED | 2 | Camera is opened and streaming has started |
| STATUS_CONNECTING | 3 | Camera is disconnected and reconnecting |
| STATUS_CONNECT_SUCCESS | 4 | Camera reconnected successfully |

## 6.22 LX_DISTORTION_MODEL

Distortion model type in the camera intrinsic parameter structure.

| Keyword | Value | Description |
|---|---|---|
| LX_DISTORTION_UNDEFINE | 0 | Invalid distortion model |
| LX_DISTORTION_RADTAN_5 | 5 | Normal radial-tangential model [k1, k2, p1, p2, k3] |
| LX_DISTORTION_RADTAN_14 | 14 | Large-distortion radial-tangential model [k1, k2, p1, p2, k3, k4, k5, k6, s1, s2, s3, s4, t1, t2] |
| LX_DISTORTION_FISHEYE | 15 | Fisheye distortion model [k1, k2, k3, k4] |
| LX_DISTORTION_SCARAMUZZA | 16 | Scaramuzza distortion model |

# 7. Feature Parameter Description

## 7.1 Int Parameters

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_INT_FIRST_EXPOSURE | 1001 | DcSetIntValue / DcGetIntValue | Default high-integration exposure value, in us. For multi-integration cases, this is the exposure time of the first integration. |
| LX_INT_SECOND_EXPOSURE | 1002 | DcSetIntValue / DcGetIntValue | Default low-integration exposure value, in us. For multi-integration |

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| | | | cases, this is the exposure time of the second integration. |
| LX_INT_THIRD_EXPOSURE | 1003 | DcSetIntValue / DcGetIntValue | For multi-integration cases, this is the exposure time of the third integration, in us. |
| LX_INT_FOURTH_EXPOSURE | 1004 | DcSetIntValue / DcGetIntValue | For multi-integration cases, this is the exposure time of the fourth integration, in us. |
| LX_INT_GAIN | 1005 | DcSetIntValue / DcGetIntValue | Gain, equivalent to the exposure effect. It introduces noise. Adjust the gain properly to avoid excessive exposure parameters. |
| LX_INT_MIN_DEPTH | 1011 | DcSetIntValue / DcGetIntValue | Minimum depth value |
| LX_INT_MAX_DEPTH | 1012 | DcSetIntValue / DcGetIntValue | Maximum depth value |
| LX_INT_MIN_AMPLITUDE | 1013 | DcSetIntValue / DcGetIntValue | Minimum amplitude value of valid signals |
| LX_INT_MAX_AMPLITUDE | 1014 | DcSetIntValue / DcGetIntValue | Maximum amplitude value of valid signals |
| LX_INT_CODE_MODE | 1016 | DcSetIntValue / DcGetIntValue | Structured-light camera coding mode. Refer to LX_STRUCT_LIGHT_CODE_MODE. |
| LX_INT_WORK_MODE | 1018 | DcSetIntValue / DcGetIntValue | Work mode. Refer to LX_CAMERA_WORK_MODE. |
| LX_INT_LINK_SPEED | 1019 | DcGetIntValue | Negotiated network adapter speed. 100 means 100 Mbps and 1000 means gigabit. Read-only. |
| LX_INT_3D_IMAGE_WIDTH | 1021 | DcGetIntValue | 3D image resolution width. It changes after ROI/Binning/2D alignment and must be obtained again. |
| LX_INT_3D_IMAGE_HEIGHT | 1022 | DcGetIntValue | 3D image resolution height. It changes after ROI/Binning/2D alignment and must be obtained again. |
| LX_INT_3D_IMAGE_OFFSET_X | 1023 | DcSetIntValue / DcGetIntValue | ROI horizontal pixel offset. WIDTH, HEIGHT, OFFSET_X, and OFFSET_Y are set together in ROI setting. Use DcSetROI to set the parameters. |

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_INT_3D_IMAGE_OFFSET_Y | 1024 | DcSetIntValue / DcGetIntValue | ROI vertical pixel offset. WIDTH, HEIGHT, OFFSET_X, and OFFSET_Y are set together in ROI setting. Use DcSetROI to set the parameters. |
| LX_INT_3D_BINNING_MODE | 1025 | DcSetIntValue / DcGetIntValue | 3D image binning. Refer to LX_BINNING_MODE. |
| LX_INT_3D_DEPTH_DATA_TYPE | 1026 | DcGetIntValue | Depth image data format. Read-only. Refer to LX_DATA_TYPE for the corresponding value. |
| LX_INT_3D_FREQ_MODE | 1027 | DcSetIntValue / DcGetIntValue | 3D frequency mode. See the definition of LX_3D_FREQ_MODE. |
| LX_INT_3D_AMPLITUDE_CHANNEL | 1031 | DcSetIntValue / DcGetIntValue | Number of 3D amplitude image channels. It shares channels with the depth image. Monochrome is 1 and color is 3. |
| LX_INT_3D_AMPLITUDE_DATA_TYPE | 1035 | DcGetIntValue | Amplitude image data format. Read-only. Refer to LX_DATA_TYPE for the corresponding value. |
| LX_INT_3D_AUTO_EXPOSURE_LEVEL | 1036 | DcSetIntValue / DcGetIntValue | Exposure level when 3D auto exposure is enabled. Exposure value and gain cannot be set during this period. |
| LX_INT_3D_AUTO_EXPOSURE_MAX | 1037 | DcSetIntValue / DcGetIntValue | Upper limit of 3D auto exposure |
| LX_INT_3D_AUTO_EXPOSURE_MIN | 1038 | DcSetIntValue / DcGetIntValue | Lower limit of 3D auto exposure |
| LX_INT_2D_IMAGE_WIDTH | 1041 | DcSetIntValue / DcGetIntValue | 2D image resolution width. It changes after ROI or Binning. |
| LX_INT_2D_IMAGE_HEIGHT | 1042 | DcSetIntValue / DcGetIntValue | 2D image resolution height. It changes after ROI or Binning. |
| LX_INT_2D_IMAGE_OFFSET_X | 1043 | DcSetIntValue / DcGetIntValue | 2D image ROI horizontal pixel offset. WIDTH, HEIGHT, OFFSET_X, and OFFSET_Y are set together in ROI setting. Use DcSetROI to set the parameters. |
| LX_INT_2D_IMAGE_OFFSET_Y | 1044 | DcSetIntValue / DcGetIntValue | 2D image ROI vertical pixel offset. WIDTH, HEIGHT, OFFSET_X, and OFFSET_Y are set together in |

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| | | | ROI setting. Use DcSetROI to set the parameters. |
| LX_INT_2D_BINNING_MODE | 1045 | DcSetIntValue / DcGetIntValue | 2D image binning. Refer to LX_BINNING_MODE. |
| LX_INT_2D_IMAGE_CHANNEL | 1046 | DcSetIntValue / DcGetIntValue | Number of 2D image channels. Monochrome and JPEG are 1, color is 3, and YUV422 is 2. |
| LX_INT_2D_IMAGE_DATA_TYPE | 1047 | DcGetIntValue | 2D image data format. Read-only. Refer to LX_DATA_TYPE for the corresponding value. |
| LX_INT_2D_IMAGE_OUTPUT_FORMAT | 1048 | DcSetIntValue / DcGetIntValue | 2D image output format. Refer to LX_2D_OUTPUT_FORMAT. |
| LX_INT_2D_ENCODE_RATIO | 1049 | DcSetIntValue / DcGetIntValue | By default, 2D images are compressed as JPEG. This parameter sets the compression ratio. |
| LX_INT_2D_MANUAL_EXPOSURE | 1051 | DcSetIntValue / DcGetIntValue | Exposure value for 2D manual exposure |
| LX_INT_2D_MANUAL_GAIN | 1052 | DcSetIntValue / DcGetIntValue | Gain value for 2D manual exposure |
| LX_INT_2D_AUTO_EXPOSURE_LEVEL | 1054 | DcSetIntValue / DcGetIntValue | Exposure level for 2D image auto exposure [0-100]. 0 means darker overall, and 100 means brighter overall. |
| LX_INT_TOF_GLOBAL_OFFSET | 1061 | DcSetIntValue / DcGetIntValue | 3D depth data offset |
| LX_INT_3D_UNDISTORT_SCALE | 1062 | DcSetIntValue / DcGetIntValue | 3D image undistortion coefficient [0,100]. The larger the value, the more field of view is retained. 0 keeps the original intrinsic parameters, and 1 means no black border. |
| LX_INT_ALGORITHM_MODE | 1065 | DcSetIntValue / DcGetIntValue | Set the built-in application algorithm. Supported by some models. Refer to LX_ALGORITHM_MODE for the corresponding value. |
| LX_INT_MODBUS_ADDR | 1066 | DcSetIntValue / DcGetIntValue | Modbus address. Some models support MODBUS protocol output through serial port. |
| LX_INT_HEART_TIME | 1067 | DcSetIntValue / DcGetIntValue | Heartbeat time with the device, in ms |

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_INT_GVSP_PACKET_SIZE | 1068 | DcSetIntValue / DcGetIntValue | GVSP single-packet data segment size, in bytes |
| LX_INT_TRIGGER_MODE | 1069 | DcSetIntValue / DcGetIntValue | Trigger mode. Refer to LX_TRIGGER_MODE for the corresponding value. |
| LX_INT_CALCULATE_UP | 1070 | DcSetIntValue / DcGetIntValue | Allows 2D or 3D algorithm computation to be migrated between the host and the camera to save computing power on either side. This may affect frame rate and latency. |
| LX_INT_CAN_BAUD_RATE | 1072 | DcSetIntValue / DcGetIntValue | CAN baud rate value, in bps |
| LX_INT_CAN_NODE_ID | 1073 | DcSetIntValue / DcGetIntValue | CAN address |
| LX_INT_CAN_PROTOCOL_TYPE | 1074 | DcSetIntValue / DcGetIntValue | CAN protocol type. Refer to LX_CAN_PROTOCOL_TYPE. |
| LX_INT_SAVE_PARAMS_GROUP | 1075 | DcSetIntValue / DcGetIntValue | Save the current camera configuration as the specified parameter group. |
| LX_INT_LOAD_PARAMS_GROUP | 1076 | DcSetIntValue / DcGetIntValue | Load a parameter group with the specified index in one step. |
| LX_INT_RGBD_ALIGN_MODE | 1077 | DcSetIntValue / DcGetIntValue | RGBD alignment mode. Refer to LX_RGBD_ALIGN_MODE. This replaces the historical compatibility item LX_BOOL_ENABLE_2D_TO_DEPTH. |
| LX_INT_XYZ_COORDINATE | 1078 | DcSetIntValue / DcGetIntValue | Point cloud coordinate system. 0: camera coordinate system, where xyz is right-down-forward. 1: robot coordinate system, where xyz is forward-left-up. |
| LX_INT_XYZ_UNIT | 1079 | DcSetIntValue / DcGetIntValue | Point cloud unit. 0: mm, 1: m. This only affects the data of LX_PTR_XYZ_DATA and LX_PTR_XYZIRT_DATA. |
| LX_INT_GVCP_TIME_OUT | 1080 | DcGetIntValue | Maximum timeout for device control and information acquisition. Default is 1800ms, in ms. |

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_INT_HARDWARE_TRIGGER_FILTER_TIME | 1085 | DcSetIntValue / DcGetIntValue | Hardware trigger filter time, in us |
| LX_INT_TRIGGER_MIN_PERIOD_TIME | 1086 | DcSetIntValue / DcGetIntValue | Minimum trigger interval, in us |
| LX_INT_TRIGGER_DELAY_TIME | 1087 | DcSetIntValue / DcGetIntValue | Trigger delay time, in us |
| LX_INT_TRIGGER_FRAME_COUNT | 1088 | DcSetIntValue / DcGetIntValue | Number of frames per trigger |
| LX_INT_FILTER_MODE | 1090 | DcSetIntValue / DcGetIntValue | Filter mode. Refer to LX_FILTER_MODE. |
| LX_INT_FILTER_SMOOTH_LEVEL | 1091 | DcSetIntValue / DcGetIntValue | When LX_INT_FILTER_MODE is FILTER_NORMAL, this sets the filter smoothing level [0, 3]. The larger the value, the stronger the filtering. |
| LX_INT_FILTER_NOISE_LEVEL | 1092 | DcSetIntValue / DcGetIntValue | When LX_INT_FILTER_MODE is FILTER_NORMAL, this sets the filter noise level [0, 3]. The larger the value, the stronger the filtering. |
| LX_INT_FILTER_TIME_LEVEL | 1093 | DcSetIntValue / DcGetIntValue | When LX_INT_FILTER_MODE is FILTER_NORMAL, this sets the temporal filter level [0, 3]. The larger the value, the stronger the filtering. |
| LX_INT_FILTER_DETECT_LOW_SIGNAL | 1094 | DcSetIntValue / DcGetIntValue | Low-signal measurement. It may detect data with a low signal-to-noise ratio, including long-distance data that causes multi-period problems. |
| LX_INT_FILTER_FILL_LEVEL | 1095 | DcSetIntValue / DcGetIntValue | When LX_INT_FILTER_MODE is FILTER_NORMAL, this sets the filter filling level [0, 3]. The larger the value, the stronger the filtering. |
| LX_INT_IMU_ACCELERATION_LEVEL | 1101 | DcSetIntValue / DcGetIntValue | IMU acceleration range level [0-3]. The larger the value, the larger the range. Each level doubles the range. |
| LX_INT_IMU_ANGULAR_RANGE_LEVEL | 1102 | DcSetIntValue / DcGetIntValue | IMU angular velocity range level [0-4]. The larger the value, the larger the range. Each level doubles the range. |

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_INT_MAX_TOLERANT_LOST_PACKETS | 1103 | DcSetIntValue / DcGetIntValue | Number of lost packets allowed in one frame. If the threshold is exceeded, the frame is discarded. Otherwise, lost packets are filled with 0. Default is 5. |
| LX_INT_3D_FPS | 1510 | DcSetIntValue / DcGetIntValue | Frame rate of depth and amplitude |
| LX_INT_2D_UNDISTORT_SCALE | 1520 | DcSetIntValue / DcGetIntValue | 2D image undistortion coefficient [0,100]. The larger the value, the more field of view is retained. 0 keeps the original intrinsic parameters, and 1 means no black border. |
| LX_INT_IO_WORK_MODE | 1530 | DcSetIntValue / DcGetIntValue | GPIO signal output control mode. Refer to LX_IO_WORK_MODE. |
| LX_INT_IO_OUTPUT_STATE | 1531 | DcSetIntValue / DcGetIntValue | User control mode of GPIO signal output. Refer to LX_IO_OUTPUT_STATE. |
| LX_INT_IO_HARDWARE_WORK_MODE | 1532 | DcSetIntValue / DcGetIntValue | Hardware work mode of GPIO signal output. Refer to LX_IO_HARDWARE_MODE. |
| LX_INT_IO_INPUT_STATUS | 1533 | DcGetIntValue | IO input status. Read-only. |
| LX_INT_IO_OUTPUT_STATUS | 1534 | DcGetIntValue | IO output status. Read-only. |
| LX_INT_3D_GLARE_LEVEL | 1738 | DcSetIntValue / DcGetIntValue | Glare level. 0 means off. Levels are 1, 2, and 3. |
| LX_INT_POWER_MODE | 1755 | DcSetIntValue / DcGetIntValue | Camera power mode. See the definition of LX_POWER_MODE_TYPE. Supported by some models. |

## 7.2 Float Parameters

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_FLOAT_FILTER_LEVEL | 2001 | DcSetFloatValue / DcGetFloatValue | When LX_INT_FILTER_MODE is FILTER_SIMPLE, this sets the filter level [0, 1]. The larger the value, the stronger the filtering. A value of 0 means filtering is disabled. |

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_FLOAT_EST_OUT_EXPOSURE | 2002 | DcSetFloatValue / DcGetFloatValue | Whether to evaluate overexposed data [0, 1]. If set to 1, overexposed data is invalid. |
| LX_FLOAT_LIGHT_INTENSITY | 2003 | DcSetFloatValue / DcGetFloatValue | Light intensity [0, 1]. Supported by some models. |
| LX_FLOAT_3D_DEPTH_FPS | 2004 | DcGetFloatValue | Current frame rate of the depth image. Read-only. |
| LX_FLOAT_3D_AMPLITUDE_FPS | 2005 | DcGetFloatValue | Current frame rate of the amplitude image. Read-only. |
| LX_FLOAT_2D_IMAGE_FPS | 2006 | DcGetFloatValue | Current frame rate of the 2D/RGB image. Read-only. |
| LX_FLOAT_DEVICE_TEMPERATURE | 2007 | DcGetFloatValue | Current camera temperature. Read-only. |
| LX_FLOAT_CONFIDENCE | 2008 | DcSetFloatValue / DcGetFloatValue | Confidence [0-1]. The smaller the value, the more data is retained. |

## 7.3 Bool Parameters

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_BOOL_CONNECT_STATE | 3001 | DcSetBoolValue / DcGetBoolValue | Current connection status |
| LX_BOOL_ENABLE_3D_DEPTH_STREAM | 3002 | DcSetBoolValue / DcGetBoolValue | Enable/disable depth data stream, supported by some cameras |
| LX_BOOL_ENABLE_3D_AMP_STREAM | 3003 | DcSetBoolValue / DcGetBoolValue | Enable/disable amplitude data stream, supported by some cameras |
| LX_BOOL_ENABLE_3D_AUTO_EXPOSURE | 3006 | DcSetBoolValue / DcGetBoolValue | Enable 3D auto exposure |
| LX_BOOL_ENABLE_3D_UNDISTORT | 3007 | DcSetBoolValue / DcGetBoolValue | Enable 3D undistortion |
| LX_BOOL_ENABLE_ANTI_FLICKER | 3008 | DcSetBoolValue / DcGetBoolValue | Enable anti-flicker. LED ambient lighting may cause obvious ripples in data. Supported by some models. |
| LX_BOOL_ENABLE_2D_STREAM | 3011 | DcSetBoolValue / DcGetBoolValue | Enable/disable 2D data stream |
| LX_BOOL_ENABLE_2D_AUTO_EXPOSURE | 3012 | DcSetBoolValue / DcGetBoolValue | Enable 2D auto exposure |
| LX_BOOL_ENABLE_2D_UNDISTORT | 3015 | DcSetBoolValue / DcGetBoolValue | Enable 2D image undistortion |

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_BOOL_ENABLE_2D_TO_DEPTH | 3016 | DcSetBoolValue / DcGetBoolValue | LX_INT_RGBD_ALIGN_MODE is recommended. |
| LX_BOOL_ENABLE_BACKGROUND_AMP | 3017 | DcSetBoolValue / DcGetBoolValue | Enable amplitude background light |
| LX_BOOL_ENABLE_MULTI_MACHINE | 3018 | DcSetBoolValue / DcGetBoolValue | Enable multi-camera mode. TOF may have multi-camera interference. Enabling this mode can automatically detect and mitigate multi-camera interference. |
| LX_BOOL_ENABLE_MULTI_EXPOSURE_HDR | 3019 | DcSetBoolValue / DcGetBoolValue | Enable HDR, multi-exposure high dynamic range mode |
| LX_BOOL_ENABLE_SYNC_FRAME | 3020 | DcSetBoolValue / DcGetBoolValue | Whether to enable forced frame synchronization. By default, data real-time performance has priority. If RGBD synchronization is required, enable this mode. |
| LX_BOOL_ENABLE_TERM_RESISTOR | 3021 | DcSetBoolValue / DcGetBoolValue | Enable termination resistor |
| LX_BOOL_ENABLE_IMU | 3023 | DcSetBoolValue / DcGetBoolValue | Enable IMU data. Obtain data through the DcRegisterImuDataCallback callback. |
| LX_BOOL_ENABLE_LASER_BRIGHT | 3110 | DcSetBoolValue / DcGetBoolValue | Enable laser on/off control |

## 7.4 String Parameters

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_STRING_DEVICE_VERSION | 4001 | DcSetStringValue / DcGetStringValue | Device version number |
| LX_STRING_FIRMWARE_NAME | 4003 | DcSetStringValue | Firmware file name, used to upgrade the device version. Some models need to reopen the camera. |
| LX_STRING_FILTER_PARAMS | 4004 | DcSetStringValue / DcGetStringValue | FILTER_NORMAL is recommended. |
| LX_STRING_ALGORITHM_PARAMS | 4005 | DcSetStringValue / DcGetStringValue | Built-in algorithm parameters. According to the currently set LX_ALGORITHM_MODE, returns the corresponding JSON format string. |

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_STRING_ALGORITHM_VERSION | 4006 | DcSetStringValue / DcGetStringValue | Built-in algorithm version number. According to the currently set LX_ALGORITHM_MODE, returns the corresponding version number. |
| LX_STRING_DEVICE_OS_VERSION | 4007 | DcSetStringValue / DcGetStringValue | Device system image version number |
| LX_STRING_IMPORT_PARAMS_FROM_FILE | 4008 | DcSetStringValue / DcGetStringValue | Load parameters from a local file to the camera |
| LX_STRING_EXPORT_PARAMS_TO_FILE | 4009 | DcSetStringValue / DcGetStringValue | Export current camera parameters to a local file |
| LX_STRING_CUSTOM_ID | 4010 | DcSetStringValue / DcGetStringValue | Customer-defined device ID identifier |

## 7.5 Command Parameters

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_CMD_GET_NEW_FRAME | 5001 | DcSetCmd | Actively update the latest current data. Call this before obtaining LX_PTR_FRAME_DATA, LX_PTR_XYZ_DATA, LX_PTR_XYZIRT_DATA, or algorithm output in synchronous streaming. It is not required in callback mode. |
| LX_CMD_RETURN_VERSION | 5002 | DcSetCmd | Roll back to the previous version |
| LX_CMD_RESTART_DEVICE | 5003 | DcSetCmd | Restart the camera. Some models need to reopen the camera. |
| LX_CMD_RESET_PARAM | 5007 | DcSetCmd | Restore default parameters |
| LX_CMD_SOFTWARE_TRIGGER | 5008 | DcSetCmd | Software trigger execution command |
| LX_CMD_GET_NEW_FRAME_3D | 5009 | DcSetCmd | Process and update only 3D data depth/amp. Use this when reducing 2D processing overhead is required. |
| LX_CMD_GET_NEW_FRAME_2D | 5010 | DcSetCmd | Process and update only 2D/RGB data. Use this when reducing 3D processing overhead is required. |

## 7.6 Ptr Parameters

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_PTR_2D_IMAGE_DATA | 6001 | DcGetPtrValue | Compatibility interface. Not recommended for priority use in new projects. Use rgb_data in LX_PTR_FRAME_DATA. |
| LX_PTR_3D_AMP_DATA | 6002 | DcGetPtrValue | Compatibility interface. Not recommended for priority use in new projects. Use amp_data in LX_PTR_FRAME_DATA. |
| LX_PTR_3D_DEPTH_DATA | 6003 | DcGetPtrValue | Compatibility interface. Not recommended for priority use in new projects. Use depth_data in LX_PTR_FRAME_DATA. |
| LX_PTR_XYZ_DATA | 6004 | DcGetPtrValue | Obtain ordinary XYZ point cloud. Float three-channel data is arranged in x, y, z order. The depth stream must be enabled and frame acquisition must be completed. |
| LX_PTR_2D_INTRIC_PARAM | 6005 | DcGetPtrValue | Compatibility interface. Use LX_PTR_2D_INTRINSIC_PARAMETERS. |
| LX_PTR_3D_INTRIC_PARAM | 6006 | DcGetPtrValue | Compatibility interface. Use LX_PTR_3D_INTRINSIC_PARAMETERS. |
| LX_PTR_3D_EXTRIC_PARAM | 6007 | DcGetPtrValue | Obtain 3D image extrinsic parameters. The pointer type is float*, and the fixed length is 12*sizeof(float). The first 9 values represent the rotation matrix and the last 3 values represent the translation vector. |
| LX_PTR_ALGORITHM_OUTPUT | 6008 | DcGetPtrValue | Compatibility interface, used to directly read the current algorithm output. The obstacle avoidance V2, pallet, and localization samples can still use it. For general frame callback scenarios, FrameInfo.app_data is preferred. |
| LX_PTR_FRAME_DATA | 6009 | DcGetPtrValue | Obtain complete one-frame data and output FrameInfo. Recommended for unified reading of depth, amplitude, RGB, and algorithm output. |

| Parameter | Number | Access Interface | Description |
|---|---|---|---|
| LX_PTR_2D_NEW_INTRIC_PARAM | 6010 | DcGetPtrValue | Use LX_PTR_2D_INTRINSIC_PARAMETERS instead. The pointer type is float*, and the fixed length is 18, including 4 intrinsic parameters and 14 distortion parameters. |
| LX_PTR_3D_NEW_INTRIC_PARAM | 6011 | DcGetPtrValue | Use LX_PTR_3D_INTRINSIC_PARAMETERS instead. The pointer type is float*, and the fixed length is 18, including 4 intrinsic parameters and 14 distortion parameters. |
| LX_PTR_2D_INTRINSIC_PARAMETERS | 6013 | DcGetPtrValue | Obtain 2D image intrinsic parameters. Refer to LxIntrinsicParameters. |
| LX_PTR_3D_INTRINSIC_PARAMETERS | 6014 | DcGetPtrValue | Obtain 3D image intrinsic parameters. Refer to LxIntrinsicParameters. |
| LX_PTR_IMU_EXTRIC_PARAM | 6015 | DcGetPtrValue | Obtain IMU image extrinsic parameters. The pointer type is float*, and the fixed length is 12*sizeof(float). The first 9 values represent the rotation matrix and the last 3 values represent the translation vector. |
| LX_PTR_XYZIRT_DATA | 6020 | DcGetPtrValue | Obtain the LiDAR point cloud structure LxPointCloudData, including timebase, point_num, and the LxPointXYZIRT point array. |

# 8. Structure Description

## 8.1 Device and Parameter Information Structures

**DcHandle**

| Field | ctypes Type | Description |
|---|---|---|
| handle | c_ulonglong | Device handle. It is used for all subsequent device operations after the device is opened successfully. |

**LxDeviceInfo**

| Field | ctypes Type | Description |
|---|---|---|
| handle | DcHandle | Unique device identifier. Subsequent interfaces after opening the device depend on this handle. |
| dev_type | c_long | Device type |
| id | c_char_Array_32 | Device ID |
| ip | c_char_Array_32 | Device IP and port |
| sn | c_char_Array_32 | Device serial number |
| mac | c_char_Array_32 | Device MAC address |
| firmware_ver | c_char_Array_32 | Device software version number |
| algor_ver | c_char_Array_32 | Device algorithm version number |
| name | c_char_Array_32 | Device name |
| reserve | c_char_Array_32 | Reserved field. Currently used for subnet mask. |
| reserve2 | c_char_Array_32 | Reserved field. Currently used for gateway IP. |
| reserve3 | c_char_Array_64 | Reserved field |
| reserve4 | c_char_Array_128 | Reserved field |

**LxIntValueInfo**

| Field | ctypes Type | Description |
|---|---|---|
| set_available | c_bool | Whether the current value can be set. true means it can be set, and false means it cannot be set. |
| cur_value | c_int | Current value |
| max_value | c_int | Maximum value |
| min_value | c_int | Minimum value |
| reserve | c_int_Array_4 | Reserved field |

**LxFloatValueInfo**

| Field | ctypes Type | Description |
|---|---|---|
| set_available | c_bool | Whether the current value can be set. true means it can be set, and false means it cannot be set. |
| cur_value | c_float | Current value |
| max_value | c_float | Maximum value |

| Field | ctypes Type | Description |
|---|---|---|
| min_value | c_float | Minimum value |
| reserve | c_float_Array_4 | Reserved field |

## 8.2 Frame Data and Camera Parameter Structures

**FrameDataInfo**

| Field | ctypes Type | Description |
|---|---|---|
| frame_data_type | c_long | Frame data type |
| frame_width | c_long | Image width |
| frame_height | c_long | Image height |
| frame_channel | c_long | Number of channels |
| frame_data | c_void_p | Image data pointer. The Python wrapper converts it to numpy.ndarray. |
| sensor_timestamp | c_ulonglong | Sensor image output timestamp |
| recv_timestamp | c_ulonglong | Timestamp when the SDK receives frame data |

**FrameInfo**

| Field | ctypes Type | Description |
|---|---|---|
| frame_state | c_long | Current frame status. First check whether it is LX_SUCCESS. |
| handle | DcHandle | Device handle |
| depth_data | FrameDataInfo | Depth image data |
| amp_data | FrameDataInfo | Amplitude image data |
| rgb_data | FrameDataInfo | 2D/RGB image data |
| app_data | FrameDataInfo | Algorithm output data. Its type is determined by the current algorithm mode. |
| reserve_data | c_void_p | Extended reserved field. The current Python samples do not directly parse this field. To obtain extended frame IDs, refer to the underlying structure definition and convert and verify it by yourself. |

**FrameExtendInfo**

| Field | ctypes Type | Description |
|---|---|---|
| depth_frame_id | c_ulong | Depth image frame ID |
| amp_frame_id | c_ulong | Amplitude image frame ID |
| rgb_frame_id | c_ulong | RGB image frame ID |
| app_frame_id | c_ulong | Application algorithm output frame ID |
| reserve_data1/2/3 | c_void_p | Reserved pointer |

**ImageIntricParam**

| Field | ctypes Type | Description |
|---|---|---|
| fx | c_float | Focal length parameter fx. |
| fy | c_float | Focal length parameter fy. |
| cx | c_float | Principal point cx. |
| cy | c_float | Principal point cy. |
| d1 | c_float | Distortion parameter 1. |
| d2 | c_float | Distortion parameter 2. |
| d3 | c_float | Distortion parameter 3. |
| d4 | c_float | Distortion parameter 4. |
| d5 | c_float | Distortion parameter 5. |

**Image3DExtricParam**

| Field | ctypes Type | Description |
|---|---|---|
| r0 | c_float | Row 1, column 1 of the rotation matrix. |
| r1 | c_float | Row 1, column 2 of the rotation matrix. |
| r2 | c_float | Row 1, column 3 of the rotation matrix. |
| r3 | c_float | Row 2, column 1 of the rotation matrix. |
| r4 | c_float | Row 2, column 2 of the rotation matrix. |
| r5 | c_float | Row 2, column 3 of the rotation matrix. |

| Field | ctypes Type | Description |
|---|---|---|
| r6 | c_float | Row 3, column 1 of the rotation matrix. |
| r7 | c_float | Row 3, column 2 of the rotation matrix. |
| r8 | c_float | Row 3, column 3 of the rotation matrix. |
| t0 | c_float | Translation vector x. |
| t1 | c_float | Translation vector y. |
| t2 | c_float | Translation vector z. |

**CameraStatus**

| Field | ctypes Type | Description |
|---|---|---|
| camera_state | c_long | Camera status |
| status_time | c_ulonglong | Status change time |
| reserve_data | c_void_p | Extended reserved field |

## 8.3 Basic Application Algorithm Structures

**LxGroundPlane**

| Field | ctypes Type | Description |
|---|---|---|
| a | c_float | Ground plane parameter a. |
| b | c_float | Ground plane parameter b. |
| c | c_float | Ground plane parameter c. |
| d | c_float | Ground plane parameter d. |

**LxPoint2d**

| Field | ctypes Type | Description |
|---|---|---|
| x | c_float | x coordinate of the 2D point. |
| y | c_float | y coordinate of the 2D point. |

**LxPoint3dWithRGB**

| Field | ctypes Type | Description |
|---|---|---|
| x | c_float | x coordinate of the 3D point. |

| Field | ctypes Type | Description |
|---|---|---|
| y | c_float | y coordinate of the 3D point. |
| z | c_float | z coordinate of the 3D point. |
| r | c_ubyte | Red component. |
| g | c_ubyte | Green component. |
| b | c_ubyte | Blue component. |

**LxVelocity**

| Field | ctypes Type | Description |
|---|---|---|
| linear | c_float_Array_3 | Linear velocity, length 3. |
| angular | c_float_Array_3 | Angular velocity, length 3. |

**LxPoseFieldDescription**

| Field | ctypes Type | Description |
|---|---|---|
| R | c_float_Array_9 | Rotation matrix, length 9. |
| T | c_float_Array_3 | Translation vector, length 3. |
| timestamp | c_ulonglong | Pose timestamp. |

## 8.4 Obstacle Avoidance Algorithm Structures

**LxObstacleBox**

| Field | ctypes Type | Description |
|---|---|---|
| pose | LxPose | Obstacle pose. |
| width | c_float | Obstacle width. |
| height | c_float | Obstacle height. |
| depth | c_float | Obstacle depth. |
| center | c_float_Array_3 | Obstacle center point. |
| type | c_long | Obstacle type. |
| id | c_longlong | Obstacle ID. |
| velocity | LxVelocity | Obstacle velocity. |

**LxObstacleBoxN**

| Field | ctypes Type | Description |
|---|---|---|
| pose | LxPose | Obstacle pose. |

| Field | ctypes Type | Description |
|---|---|---|
| width | c_float | Obstacle width. |
| height | c_float | Obstacle height. |
| depth | c_float | Obstacle depth. |
| center | c_float_Array_3 | Obstacle center point. |
| type_idx | c_long | Obstacle type index. |
| id | c_longlong | Current obstacle ID. |
| prev_id | c_longlong | Associated ID of the previous frame. |
| box_2d_x_min | c_float | Minimum x value of the 2D detection box. |
| box_2d_y_min | c_float | Minimum y value of the 2D detection box. |
| box_2d_x_max | c_float | Maximum x value of the 2D detection box. |
| box_2d_y_max | c_float | Maximum y value of the 2D detection box. |
| velocity | LxVelocity | Obstacle velocity. |
| type_name_len | c_long | Type name length. |
| type_name | c_char_Array_512 | Type name. |
| reserved | c_char_Array_1024 | Reserved field. |

**LxAvoidanceOutput**

| Field | ctypes Type | Description |
|---|---|---|
| state | c_long | Obstacle avoidance algorithm status. |
| groundPlane | LxGroundPlane | Ground plane parameter . |
| number_3d | c_ulong | Number of output points. |
| cloud_output | LP_LxPoint3dWithRGB | Pointer to 3D point cloud with RGB. |
| number_box | c_ulong | Number of obstacle boxes. |
| obstacleBoxs | LP_LxObstacleBox | Pointer to obstacle box array. |

**LxAvoidanceOutputN**

| Field | ctypes Type | Description |
|---|---|---|
| state | c_int | Obstacle avoidance V2 algorithm status. |
| groundPlane | LxGroundPlane | Ground plane parameter . |
| number_3d | c_uint32 | Number of output points. |
| cloud_output | POINTER(LxPoint3dWithRGB) | Pointer to 3D point cloud with RGB. |
| number_box | c_uint32 | Number of obstacle boxes. |
| obstacleBoxs | POINTER(LxObstacleBoxN) | Pointer to obstacle avoidance V2 obstacle box array |
| resveredData | c_char_Array_4096 | Reserved field. The source field name is resveredData. |

## 8.5 Pallet and Vision Localization Structures

**LxPalletPose**

| Field | ctypes Type | Description |
|---|---|---|
| return_val | c_long | Return value of the pallet algorithm. |
| x | c_float | Pallet pose x. |
| y | c_float | Pallet pose y. |
| yaw | c_float | Pallet heading angle. |
| extents | c_long_Array_8192 | Extended field. |

**LxRelocPose**

| Field | ctypes Type | Description |
|---|---|---|
| x | c_double | Relocalization pose x. |
| y | c_double | Relocalization pose y. |
| theta | c_double | Relocalization angle. |

**LxOdomData**

| Field | ctypes Type | Description |
|---|---|---|
| timestamp | c_longlong | Odometry timestamp. |
| x | c_double | Odometry x. |
| y | c_double | Odometry y. |

| Field | ctypes Type | Description |
|---|---|---|
| theta | c_double | Odometry angle. |

**LxLaser**

| Field | ctypes Type | Description |
|---|---|---|
| timestamp | c_longlong | Laser data timestamp. |
| time_increment | c_float | Time interval between scan points. |
| angle_min | c_float | Minimum angle. |
| angle_max | c_float | Maximum angle. |
| angle_increment | c_float | Angle increment. |
| range_min | c_float | Minimum range. |
| range_max | c_float | Maximum range. |
| reserved | c_float | Reserved field. |
| range_size | c_float | Length of the range array. |
| ranges | LP_c_float | Pointer to the range array. |

**LxLaserPose**

| Field | ctypes Type | Description |
|---|---|---|
| timestamp | c_longlong | Laser pose timestamp. |
| x | c_double | Laser pose x. |
| y | c_double | Laser pose y. |
| theta | c_double | Laser pose angle. |

**LxLocation**

| Field | ctypes Type | Description |
|---|---|---|
| timestamp | c_longlong | Localization result timestamp. |
| status | c_long | Localization status. |
| x | c_double | Localization x. |
| y | c_double | Localization y. |
| theta | c_double | Localization angle. |
| extents | c_long_Array_8192 | Extended field. |

**LxAvState**

| Name | Value | Description |
|---|---|---|
| LxAvSuccess | 0 | Obstacle avoidance algorithm normal. |
| LxAvGroundPriorNotSet | -1 | Ground prior not set. |
| LxAvInputNotSet | -2 | Input not set. |
| LxAvNoPointInRange | -3 | No point in range. |
| LxAvNoPointAfterFilterGround | -4 | No point after filtering ground. |
| LxAvNoPointAfterRadiusFilter | -5 | No point after radius filtering. |
| LxAvObstaclesNotSegmented | -6 | Obstacles not segmented. |
| LxAvUndefinedError | -99 | Undefined error. |

# 9. API Description

## 9.1 Version, Log, and Global Configuration

| Interface | Return Value | Description |
|---|---|---|
| DcGetApiVersion() | str | Gets the version of the currently loaded underlying API. |
| DcSetInfoOutput(print_level,enable_screen_print,log_path,language=0) | LX_STATE | Sets the log level, whether to output to the screen, log directory, and language. print_level is 0/1/2, corresponding to all, warning, and error. |
| DcLog(string) | LX_STATE | Outputs custom debugging information to the SDK log. |
| DcGetErrorString(state) | str | Gets the underlying error description according to LX_STATE. The parameter must be an LX_STATE object. |

## 9.2 Finding, Connecting, and Closing Cameras

| Interface | Return Value | Description |
|---|---|---|
| DcGetDeviceList() | LX_STATE,dev_list,dev_num | Searches the camera list. dev_list is a pointer array of LxDeviceInfo, and dev_num is the quantity. |
| DcOpenDevice(open_mode,param) | LX_STATE,DcHandle,LxDeviceInfo | Opens the device in the specified mode. open_mode must be LX_OPEN_MODE. |

| Interface | Return Value | Description |
|---|---|---|
| DcCloseDevice(handle) | LX_STATE | Closes the device and releases control permission. |

## 9.3 Starting and Stopping Streaming

| Interface | Return Value | Description |
|---|---|---|
| DcStartStream(handle) | LX_STATE | Starts data streaming. Parameters that must be set before streaming starts should be configured before this call. |
| DcStopStream(handle) | LX_STATE | Stops data streaming. It is recommended to stop streaming before closing the device. |

## 9.4 Reading and Setting Camera Parameters

| Interface | Return Value | Description |
|---|---|---|
| DcSetIntValue(handle,cmd,value) | LX_STATE | Sets an LX_INT parameter. value can be int or Enum. |
| DcGetIntValue(handle,cmd) | LX_STATE,LxIntValueInfo | Reads an LX_INT parameter and its range information. |
| DcSetFloatValue(handle,cmd,value) | LX_STATE | Sets an LX_FLOAT parameter. |
| DcGetFloatValue(handle,cmd) | LX_STATE,LxFloatValueInfo | Reads an LX_FLOAT parameter and its range information. |
| DcSetBoolValue(handle,cmd,value) | LX_STATE | Sets an LX_BOOL parameter. |
| DcGetBoolValue(handle,cmd) | LX_STATE,bool | Reads an LX_BOOL parameter. |
| DcSetStringValue(handle,cmd,value) | LX_STATE | Sets an LX_STRING parameter. The string is passed to the underlying library in UTF-8. |
| DcGetStringValue(handle,cmd) | LX_STATE,str | Reads an LX_STRING parameter. An empty string is returned on failure. |
| DcSetCmd(handle,cmd) | LX_STATE | Executes an LX_CMD command. |

## 9.5 Reading Images, Point Clouds, and Camera Parameters

| Interface | Return Value | Description |
|---|---|---|
| getFrame(handle) | LX_STATE,FrameInfo | Internally executes LX_CMD_GET_NEW_FRAME and returns FrameInfo after success. |
| getRGBImage(frame) | LX_STATE,numpy.ndarray | Reads the RGB image from FrameInfo.rgb_data. The shape is (height,width,channel). |
| getDepthImage(frame) | LX_STATE,numpy.ndarray | Reads the depth image from FrameInfo.depth_data. The shape is (height,width). |
| getAmpImage(frame) | LX_STATE,numpy.ndarray | Reads the amplitude image from FrameInfo.amp_data. The shape is (height,width). |
| getPointCloud(handle) | LX_STATE,numpy.ndarray | Reads the XYZ point cloud. The shape is (depth_height, depth_width,3). The 3D depth stream must be enabled and a frame must be refreshed before calling it. |
| DcSaveXYZ(handle,filename) | LX_STATE | Saves point clouds. Supports txt, ply, and pcd. txt saves all data in image order. ply/pcd usually saves only non-zero data. |
| get2DIntricParam(handle) | LX_STATE,intrinsic,distort | Reads 2D camera intrinsic parameters. intrinsic is [fx,fy,cx,cy], and distort is [d1,d2,d3,d4,d5]. |
| get3DIntricParam(handle) | LX_STATE,intrinsic,distort | Reads 3D camera intrinsic parameters. |
| get3DTransMatrix(handle) | LX_STATE,numpy.ndarray | Reads 3D extrinsic parameters and returns a matrix with shape (3,4). The first 3 columns are the rotation matrix, and the fourth column is the translation vector. |

## 9.6 Special Control, ROI, and Algorithm Output

| Interface | Return Value | Description |
|---|---|---|
| DcSetCameraIp(handle,ip,netmask='',gateway='') | LX_STATE | Sets the camera IP, subnet mask, and gateway. The camera restarts automatically after setting. |

| Interface | Return Value | Description |
|---|---|---|
| DcSpecialControl(handle,command,value=None) | LX_STATE,Any | Calls extended control commands, such as GetObstacleIO, SetLaserData, and ImportLocationMapFile. |
| DcSetRoI(handle,offset_x,offset_y,width,height,img_type) | LX_STATE | Sets ROI. img_type 0 means 3D image, and 1 means 2D image. The uppercase/lowercase spelling of the underlying function in the current source code must be confirmed. |
| getAlgorithmStatus(handle) | LX_STATE,Any | Returns the corresponding algorithm structure according to LX_INT_ALGORITHM_MODE. |

## 9.7 Status Callback

| Interface | Return Value | Description |
|---|---|---|
| DcRegisterCameraStatusCallback(handle,func) | LX_STATE | Registers a camera status callback. func receives a CameraStatus pointer. |
| DcUnregisterCameraStatusCallback(handle) | LX_STATE | Unregisters the camera status callback. |

# 10. Sample Program Description

## 10.1 Sample Program Index

| Sample | Main Function | Current Status |
|---|---|---|
| single_camera.py | Single-camera synchronous streaming, status callback, 2D/3D stream switches, image display, point cloud reading, and frame rate reading. | Recommended as the main reference sample for basic Python development. |
| application_obstacle.py | Switches obstacle avoidance V2 algorithm, reads and writes algorithm parameters, and reads GetObstacleIO and algorithm output after streaming starts. | The sample process can be referenced. If the current lx_camera_api.py interface is used, adjust the device search part to state,dev_list,dev_num = camera.DcGetDeviceList(). |

| Sample | Main Function | Current Status |
|---|---|---|
| application_pallet.py | Switches pallet algorithm, reads and writes algorithm parameters, and reads x, y, and yaw in LxPalletPose. | The device search code in the sample must be corrected according to the current return values of DcGetDeviceList. |
| application_location.py | Reference for vision localization-related structures and commands such as SetLaserData and ImportLocationMapFile. | The source code is marked TODO NOT FINISHED, DO NOT USE IT. It must be completed and verified before use in formal projects. |
| utils.py | Image normalization display and error status checking. | It can be copied into a project and adjusted as needed. |

## 10.2 single_camera.py: Single-camera Synchronous Streaming

single_camera.py is the most complete basic sample in the current Python package. It selects the open mode by IP, ID, SN, or index, registers a status callback, enables 2D, 3D depth, and 3D amplitude streams, reads image size and data type, calls getFrame in a loop after streaming starts, and converts image data through getDepthImage, getAmpImage, and getRGBImage.

| Stage | Key Interface | Description |
|---|---|---|
| Search devices | DcGetDeviceList | Returns state, dev_list, and dev_num. Exit when dev_num<=0. |
| Open device | DcOpenDevice | The sample opens by IP by default. It can be switched to SN, ID, or index. |
| Status monitoring | DcRegisterCameraStatusCallback | Unregisters the status callback after testing it. In actual projects, registration can be kept until before the device is closed. |
| Enable data streams | DcSetBoolValue | Controls LX_BOOL_ENABLE_2D_STREAM, LX_BOOL_ENABLE_3D_DEPTH_STREAM, and LX_BOOL_ENABLE_3D_AMP_STREAM respectively. |
| Read image information | DcGetIntValue | Reads width, height, channel count, and data type. |
| Refresh frame | getFrame | Refreshes FrameInfo in the loop. |
| Image conversion | getDepthImage/getAmpImage/getRGBImage | Returns numpy.ndarray and can be displayed with OpenCV. |
| Point cloud | getPointCloud | Reads the XYZ point cloud. The sample uses open3d for visualization. |

## 10.3 application_obstacle.py: Obstacle Avoidance Algorithm

The obstacle avoidance sample sets LX_INT_ALGORITHM_MODE to MODE_AVOID_OBSTACLE2, reads the algorithm version and parameter JSON, and writes parameters back after modification when necessary. After streaming starts, it calls getFrame in a loop, reads IO results through DcSpecialControl(handle,'GetObstacleIO'), and reads LxAvoidanceOutputN through getAlgorithmStatus.

## 10.4 application_pallet.py: Pallet Docking Algorithm

The pallet sample sets LX_INT_ALGORITHM_MODE to MODE_PALLET_LOCATE, reads algorithm parameters, enters the streaming loop, and obtains LxPalletPose through getAlgorithmStatus. Commonly used fields are x, y, and yaw, which represent pallet pose output.

## 10.5 application_location.py: Vision Localization Reference

The vision localization sample is currently unfinished, and the source code clearly marks it as not recommended for direct use. This file can still be used to understand the calling direction of structures and control commands such as LxLaser, SetLaserData, and ImportLocationMapFile. In formal projects, first confirm the algorithm mode, map file format, laser data lifecycle, and DcSpecialControl parameter passing method. In addition, the algorithm mode set in the current application_location.py is not MODE_VISION_LOCATION, but MODE_AVOID_OBSTACLE2. Therefore, this file can only be used as a reference for structures and DcSpecialControl calling methods, and cannot be used directly as a complete vision localization sample.

## 10.6 utils.py: Error Handling and Image Display

The visualize function in utils.py normalizes images to 0 to 255 and applies OpenCV pseudo-color. It is suitable for debugging depth images and amplitude images. check_state outputs the error string when the state is not LX_SUCCESS and closes the device when handle is passed in. In a project, it can be changed to raise exceptions or record logs according to the project's own exception handling method.

# 11. Debugging and FAQs

Table 11-1 Common Problem Troubleshooting

| Problem | Most Likely Cause | Handling Method |
| --- | --- | --- |
| LxCamera initialization fails | dll_path is incorrect, the dynamic library suffix does not match the platform, or the | Confirm that the file exists. Use .dll on Windows and .so on Linux. 64-bit |

| Problem | Most Likely Cause | Handling Method |
|---|---|---|
| | Python architecture does not match the DLL architecture. | Python loads the win_x64 dynamic library. |
| Camera cannot be found | The host and camera are not on the same subnet, the firewall blocks traffic, or the Ethernet cable or power supply is abnormal. | Check the 192.168.100.* subnet, disable or allow through the firewall, and confirm network port negotiation speed and power supply. |
| DcOpenDevice fails | open_mode and param do not match, IP/SN/ID is incorrect, or camera control permission is occupied. | Use DcGetDeviceList to confirm device information. Close other programs occupying the camera. Retry after the heartbeat times out. |
| getFrame returns non-success | Streaming is not started, the network is abnormal, frame synchronization times out, or multi-camera interference exists. | Confirm that DcStartStream succeeds. For LX_E_RECONNECTING, wait for reconnection. For LX_E_FRAME_ID_NOT_MATCH and LX_E_FRAME_MULTI_MACHINE, handle them according to on-site synchronization and multi-camera mode. |
| Image pointer is null | The corresponding data stream is not enabled, or no valid frame is available after streaming starts. | Set the corresponding LX_BOOL_ENABLE_*_STREAM to True first, then start streaming and call getFrame. |
| getPointCloud fails | 3D depth stream is not enabled, or no new frame has been refreshed. | Enable LX_BOOL_ENABLE_3D_DEPTH_STREAM first, and call getFrame or DcSetCmd(LX_CMD_GET_NEW_FRAME). |
| Algorithm parameter reading fails | LX_INT_ALGORITHM_MODE was not set first, or the current device does not support the algorithm. | Set the algorithm mode first, and then read LX_STRING_ALGORITHM_VERSION and LX_STRING_ALGORITHM_PARAMS. |
| DcSetRoI is unavailable | The Python wrapper and the underlying symbol name use inconsistent uppercase/lowercase spelling, or the device/current state does not allow ROI setting. | Confirm the exported name of the dynamic library. Fix the wrapper if necessary. Set ROI after stopping streaming, and read image width and height again. |
| OpenCV window does not display | opencv-python is not installed, or the runtime environment does not support GUI display. | Install opencv-python. In a headless environment, only save images or print shape. |

# 12. Version and Compatibility Notes

Table 12-1 Version and Compatibility Notes

| Item | Description |
| --- | --- |
| Python package version | The current whl is lx-camera-py 1.3.3. The source package dist-info declares dependencies on numpy and opencv-python. |
| Wrapper version check | SDK_VERSION in lx_camera_api.py is 2.4.30. When LxCamera is initialized, DcGetApiVersion is called. If the loaded version is less than or equal to this value, a log is written and a version mismatch prompt is printed. |
| Underlying SDK version | The current version macro in SDK/include/lx_camera_api_version.h is LX_CAMERA_API_VERSION V2.4.60,20260126. The actual runtime version depends on the dynamic library loaded by LxCamera. |
| Platform support | The Python wrapper determines Windows and Linux according to sys.platform. Other platforms throw NotImplementedError. |
| Interface coverage | The current Python wrapper does not cover all interfaces in the C/C++ guide. For example, frame callbacks, IMU callbacks, GPU/PTP/JPEG decoding configuration, and others are not wrapped in the LxCamera class. If these capabilities are needed, extend the ctypes wrapper or use the C/C++ interface. |
| Sample status | single_camera.py is relatively complete. application_location.py is still an unfinished sample. Some algorithm samples must be corrected according to the current return values of DcGetDeviceList before running. |
| Items to confirm | The underlying ROI function naming, DcSpecialControl command parameters for application algorithms, vision localization map file format, and laser data passing method should be verified on the specific device and algorithm version. |